

รายงานปฏิบัติการ Lab 8: Table Relationships — Product Shop

วิชา CP353002 Principles of Software Design

ชื่อ - นามสกุล: แทนคุณ พันธนิกุล

รหัสนักศึกษา: 673380301-0

กลุ่มเรียน (Section): SEC 1

GitHub Repository: https://github.com/tankeiei/673380301-0-LAB08_Table_Relationships

ส่วนที่ 1: หลักการออกแบบ (Software Design Principles)

1.1 การประยุกต์ใช้ SOLID Principles กับ Entity และ สถาปัตยกรรมซอฟต์แวร์

1. S — Single Responsibility Principle (SRP)

- การออกแบบ Entity: แยกตาราง ProductDetail และ Review ออกจาก Product เพื่อให้แต่ละ Entity จัดการข้อมูลเฉพาะด้านของตนเอง ไม่ทำให้ตาราง Product บวม

- การออกแบบ Architecture: แยก ProductService ให้จัดการเฉพาะ Business Logic และ ProductController ให้จัดการเฉพาะการรับ-ส่ง HTTP Request เท่านั้น

2. O — Open/Closed Principle (OCP)

- การออกแบบ Relationship: ออกแบบความสัมพันธ์แบบ 1:N กับ Review ทำให้สามารถเพิ่มความคิดเห็นใหม่ๆ ได้ไม่จำกัด โดยไม่ต้องแก้ไขโครงสร้างตาราง Product

- การออกแบบ Strategy Pattern: การคำนวณส่วนลดผ่าน DiscountStrategy ช่วยให้สามารถเพิ่มโปรโมชันใหม่ เช่น VIP ได้โดยการสร้าง Class ใหม่ที่ implement DiscountStrategy โดยไม่ต้องแก้ไขโค้ดคำนวณเดิม

3. L — Liskov Substitution Principle (LSP)

- Concrete Strategies ทุกตัว (NoDiscountStrategy, MemberDiscountStrategy, SeasonalSaleStrategy) สามารถถูกนำไปแทนที่ Interface DiscountStrategy ใน Service ได้อย่างไรข้อผิดพลาด

- ProductRepository สืบทอด JpaRepository จึงสามารถเรียกใช้เมธอดมาตรฐาน (findAll, save, delete) ได้อย่างสมบูรณ์

4. I — Interface Segregation Principle (ISP)

- แยก Repository ออกเป็น ProductRepository, ProductDetailRepository, ReviewRepository จัดเจนตาม Entity แทนการรวมเป็น Repository ขนาดใหญ่ตัวเดียว

- Interface DiscountStrategy มีเฉพาะเมธอด calculateDiscountedPrice() เท่าที่จำเป็น ไม่ผูกมัดเมธอดที่ไม่เกี่ยวข้อง

5. D — Dependency Inversion Principle (DIP)

- ProductService ขึ้นอยู่กับ ProductRepository (Interface) โดยไม่ผูกติดกับการ Implement ฐานข้อมูลจริงโดยตรง

- ใช้ Constructor Injection ใน Controller และ Service เพื่อฉีด Dependency เข้ามา แทนการสร้าง instance ด้วย new ภายในคลาส

1.2 ความแตกต่างของความสัมพันธ์ One-to-One (1:1) และ One-to-Many (1:N)

• One-to-One (1:1) [Product (1) <—> (1) ProductDetail]

- สินค้า 1 ชิ้น มีรายละเอียดเสริม 1 รายการ

- ตำแหน่ง Foreign Key: อยู่ที่ฝั่ง Owner คือตาราง products จะมีคอลัมน์ detail_id ชี้ไปยัง id ของ product_details

- Annotations: ฝั่ง Owner ใช้ OneToOne + JoinColumn(name="detail_id") และฝั่ง Inverse ใช้ OneToOne(mappedBy="detail")

- เหตุผลที่ใช้: ข้อมูลเสริมเฉพาะด้านที่ไม่ได้ถูกเรียกดูบ่อย (เช่น น้ำหนัก, ขนาด, การรับประกัน) แยกออกมาเพื่อลดภาระการ Query ข้อมูลสินค้าหลัก

- Lifecycle: ใช้ CascadeType.ALL เพื่อให้การบันทึกหรือลบสินค้า จัดการรายละเอียด ProductDetail ควบคู่กันไปทันที

• One-to-Many (1:N) [Product (1) <—> (N) Review]

- สินค้า 1 ชิ้น มีผู้รีวิวได้หลายรายการ

- ตำแหน่ง Foreign Key: อยู่ที่ฝั่ง Many เสมอ คือตาราง reviews จะมีคอลัมน์ product_id ชี้กลับมายัง id ของ products

- Annotations: ฝั่ง One ใช้ OneToMany(mappedBy="product") และฝั่ง Many ใช้ ManyToOne + JoinColumn(name="product_id")
- เหตุผลที่ใช้: ใช้กับข้อมูลรายการย่อย ที่มีจำนวนเพิ่มขึ้นได้เรื่อยๆ โดยไม่จำกัด เพื่อให้เป็นไปตามหลัก Database Normalization
- Lifecycle: ใช้ CascadeType.ALL ร่วมกับ orphanRemoval = true เพื่อให้เมื่อลบสินค้า รีวิวทั้งหมดของสินค้านั้นจะถูกลบตามอัตโนมัติ

1.3 การประยุกต์ใช้ Strategy Pattern สำหรับการคำนวณส่วนลด

ระบบได้ออกแบบการคำนวณส่วนลดสินค้าตามประเภท (discountType) โดยใช้ Strategy Pattern ซึ่งประกอบด้วย 3 องค์ประกอบหลัก:

1. Strategy Interface (DiscountStrategy) กำหนดสัญญาด้วยเมธอด calculateDiscountedPrice(double originalPrice)

2. Concrete Strategies (Class ลูกที่ Implement อัลกอริทึม)

- NoDiscountStrategy: สำหรับส่วนลด NONE คำนวณราคาปกติ ไม่มีการลด (ลด 0%)
- MemberDiscountStrategy: สำหรับส่วนลด MEMBER คำนวณลด 10% ($\text{originalPrice} * 0.90$)
- SeasonalSaleStrategy: สำหรับส่วนลด SEASONAL คำนวณลด 20% ($\text{originalPrice} * 0.80$)

3. Strategy Context / Factory (DiscountContext) มีเมธอด getStrategy(discountType) ทำหน้าที่เลือกและส่งคืน Strategy ที่ถูกต้องตามประเภทส่วนลด

- ประโยชน์: ช่วยจัดโค้ดเงื่อนไข if-else หรือ switch-case ที่ซับซ้อนใน Service Layer ทำให้โค้ดอ่านง่าย สามารถแยกทดสอบการคำนวณแต่ละแบบได้อิสระ และรองรับการเพิ่มโปรโมชั่นใหม่ได้อย่างง่ายดายตามหลัก OCP

1.4 Execution Flow การทำงานของระบบตั้งแต่ HTTP Request สู Database

ลำดับขั้นตอนการทำงานของระบบในการประมวลผลคำร้องขอ (เช่น การบันทึกสินค้าใหม่ผ่าน POST /products/save)

1. Client Layer: ผู้ใช้งานกรอกข้อมูลสินค้า สเปก และรีวิวแรกผ่านหน้าเว็บ /products/add แล้วกด Submit เพื่อส่ง HTTP POST ไปยัง Server
2. Controller Layer: Spring DispatcherServlet รับ Request และส่งต่อให้ ProductController.saveProduct() พร้อมทำ Data Binding ข้อมูลฟอร์มเข้าสู่ Product Object
3. Service Layer: Controller เรียกใช้ ProductService.saveProduct() ซึ่งในขั้นตอนนี้ Service จะจัดการผูกความสัมพันธ์สองทิศทาง โดยตั้งค่า detail.setProduct(product) และกำหนดความสัมพันธ์ให้ Review ก่อนบันทึก
4. Strategy Calculation: เมื่อมีการเรียกดูข้อมูลสินค้า (GET /products) Service จะเรียก DiscountContext เพื่อคำนวณ discountedPrice และกำหนดให้กับสินค้าแต่ละชิ้นก่อนส่งไปแสดงผล
5. Repository Layer (JPA / Hibernate): Service เรียก ProductRepository.save(product) โดย Spring Data JPA จะแปลง Object เป็นคำสั่ง SQL (INSERT INTO products, product_details, reviews) อัตโนมัติ

6. Database Layer (PostgreSQL): ฐานข้อมูลทำการบันทึกข้อมูลลงใน 3 ตารางจริง พร้อมจัดการตรวจสอบความถูกต้องของ Foreign Key Constraints

7. View Rendering: Controller สั่ง Redirect กลับมายังหน้า /products พร้อมแนบ Flash Message และ Thymeleaf ทำการเรนเดอร์ข้อมูลสินค้าออกมาเป็น HTML ส่งกลับให้ Browser

ส่วนที่ 2: Code และคำอธิบาย (Code Implementation & Explanation)

2.1 Entity ทั้ง 3 คลาส และการอธิบาย JPA Annotations

- Entity & Table: ระบุว่าคลาสนี้เป็น JPA Entity ที่แมปกับตารางใน Database และกำหนดชื่อตาราง เช่น `Table(name = "products")`
- Id & GeneratedValue: กำหนด Primary Key และระบุให้ PostgreSQL สร้างค่า ID แบบ Auto-increment อัตโนมัติด้วย `GenerationType.IDENTITY`
- OneToOne(cascade = ALL): กำหนดความสัมพันธ์ 1:1 กับ `ProductDetail` โดย `CascadeType.ALL` ช่วยให้การบันทึก/อัปเดต/ลบ `Product` จะส่งผลต่อไปยัง `ProductDetail` อัตโนมัติ
- JoinColumn(name = "detail_id"): ระบุว่าคอลัมน์ `detail_id` ในตาราง `products` เป็น Foreign Key ที่ชี้ไปยัง id ของตาราง `product_details` (ฝั่ง Owner)
- OneToOne(mappedBy = "detail"): กำหนดความสัมพันธ์ 1:1 ฝั่ง Inverse บน `ProductDetail` โดย `mappedBy` ชี้ไปที่ตัวแปร `detail` ใน `Product` เพื่อป้องกันไม่ให้สร้าง Foreign Key ซ้ำซ้อน
- OneToMany(mappedBy = "product"): กำหนดความสัมพันธ์ 1:N กับ `Review` โดย `mappedBy="product"` ระบุว่าฝั่ง `Review` เป็นผู้ถือ Foreign Key
- ManyToOne & JoinColumn(name = "product_id"): กำหนดความสัมพันธ์ N:1 กับ `Product` และระบุคอลัมน์ Foreign Key `product_id` ไว้ในตาราง `reviews` (ฝั่ง Many เป็นผู้เก็บ FK เสมอ)
- Transient: ระบุว่าฟิลด์ `discountedPrice` ใช้สำหรับเก็บราคาหลังหักส่วนลดเพื่อแสดงผลบนหน้าเว็บเท่านั้น ไม่ต้องสร้างเป็นคอลัมน์จริงใน Database

2.2 Service และ Controller พร้อมอธิบาย Constructor Injection (DIP)

ในการพัฒนาสถาปัตยกรรม Spring Boot ของ Lab นี้ ทั้ง ProductService และ ProductController ได้ถูกออกแบบให้ใช้ Constructor Injection ในการรับ Dependencies แทนการใช้ @Autowired บนฟิลด์โดยตรง เช่น:

```
private final ProductRepository productRepository;

public ProductService(ProductRepository productRepository) {

    this.productRepository = productRepository;

}
```

เหตุผลและประโยชน์ตามหลักการออกแบบซอฟต์แวร์ (DIP):

1. Immutability: สามารถประกาศตัวแปร Reference เป็น final ได้ ทำให้มั่นใจได้ว่า Dependency จะไม่ถูกเปลี่ยนแปลงหรือกำหนดค่าใหม่ภายหลังการสร้าง Object
2. Compile-Time Safety: ป้องกันการเกิด NullPointerException ในขณะรันโปรแกรม เนื่องจาก Spring หรือผู้เรียกใช้จำเป็นต้องส่ง Dependency ที่ครบถ้วนเข้ามาตั้งแต่ตอน Instantiate
3. Ease of Unit Testing: ทำให้สามารถเขียนชุดทดสอบ (Unit Test) ได้สะดวก โดยสามารถส่ง Mock Repository เข้าไปทาง Constructor ได้โดยตรงโดยไม่ต้องอาศัยการโหลด Spring Application Context

ภาพที่ 1 หน้าฟอร์มเพิ่มสินค้าใหม่

ภาพที่ 1 หน้าฟอร์มเพิ่มสินค้าใหม่

ภาพที่ 2 หน้ารายการสินค้าในตาราง

Product Shop

Lab 8 — 1:1 & 1:N Relationship

เพิ่มสินค้าเรียบร้อยแล้ว!

+ เพิ่มสินค้าใหม่

#	ชื่อสินค้า	หมวดหมู่	ยี่ห้อ	คลัง	ราคา (บาท)	ส่วนลด	ราคาสุทธิ	รับประกัน (1:1)	น้ำหนัก (1:1)	รวม (1:N)	จัดการ
1	iPhone 15 Pro (256 GB)	Electronics	Apple	4	45900.00	MEMBER	41310.00	1 year 8 month	0.85	0 ชิ้น	แก้ไขลบ
2	Oppo find X5 (673380301-0 Sec 1)	Electronics	Oppo	4	39900.00	SEASONAL	31920.00	2 year	0.3	1 ชิ้น	แก้ไขลบ

ภาพที่ 3 หน้าฟอร์มแก้ไขสินค้า

แก้ไขสินค้า

ข้อมูลสินค้า (Product)

ชื่อสินค้า

Oppo find X5 (673380301-0 Sec 1)

หมวดหมู่

Electronics

ยี่ห้อ

Oppo

จำนวนในคลัง

4

ราคาปกติ (บาท)

39900.0

ประเภทส่วนลด

ส่วนลดเทศกาล (20%)

ข้อมูลเสริมสินค้า (ProductDetail) — ความสัมพันธ์ 1:1

รายละเอียดสินค้า

ของแท่นนี้

ระยะเวลาประกัน

2 year

น้ำหนัก (กิโลกรัม)

0.3

ขนาด

15×7×0.8 cm

ประเทศที่ผลิต

China

บันทึกการแก้ไข

ยกเลิก

ภาพที่ 4 หน้ายืนยันการลบสินค้า

ยืนยันการลบสินค้า

คุณต้องการลบสินค้านี้ใช่หรือไม่? การดำเนินการนี้ไม่สามารถย้อนกลับได้

ชื่อสินค้า:

iPhone 15 Pro (256 GB)

หมวดหมู่:

Electronics

ยี่ห้อ:

Apple

ราคา:

45900.00 บาท

รายละเอียด:

รับประกัน 1 ปี 6 เดือน, น้ำหนัก 1.61 กก., ขนาด 15×7×0.8 ซม., ผลิตจาก China / USA / Japan

รับประกัน:

1 year 8 month

⚠ ข้อมูลเสริม (ProductDetail) จะถูกลบพร้อมกันด้วย เนื่องจาก CascadeType.ALL

ยืนยันลบ

ยกเลิก

ภาพที่ 5 หน้าจอหลังลบสินค้าสำเร็จ

Product Shop

Lab 8 — 1:1 & 1:N Relationship

ลบสินค้าสำเร็จ!

+ เพิ่มสินค้าใหม่

#	ชื่อสินค้า	หมวดหมู่	ยี่ห้อ	คลัง	ราคา (บาท)	ส่วนลด	ราคาสุทธิ	รับประกัน (1:1)	น้ำหนัก (1:1)	รีวิว (1:N)	จัดการ
2	Oppo find X5 (673380301-0 Sec 1)	Electronics	Oppo	4	39900.00	SEASONAL	31920.00	2 year	0.3	1 รีวิว	แก้ไขลบ

ภาพที่ 6 ฐานข้อมูล PostgreSQL (pgAdmin 4) — ตาราง products

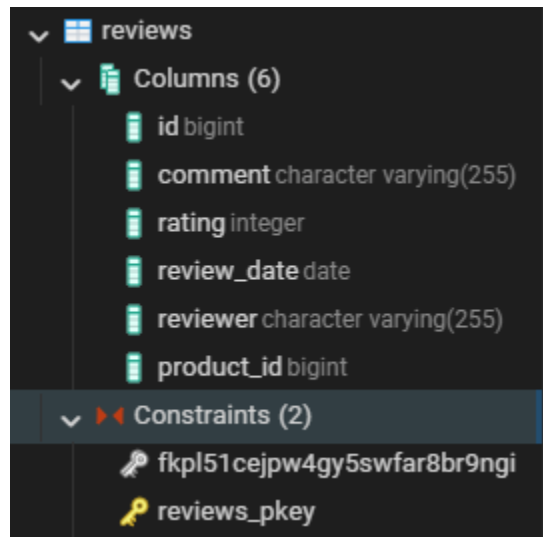
The screenshot shows the pgAdmin 4 interface. On the left, the 'products' table is selected, showing its columns and constraints. The columns are: id (bigint), brand (character varying(255)), category (character varying(255)), discount_type (character varying(255)), name (character varying(255)), price (double precision), stock (integer), and detail_id (bigint). The constraints are: fka9j512fdisl3ol4ix883w8bcj, products_pkey, and ukfq6xkbp2dny2wua6wcu5jcwj. On the right, a SQL query is executed, showing the result of a SELECT statement.

```
(C) Microsoft Corporation. All rights reserved.  
C:\Program Files\PostgreSQL\18\pgAdmin 4\runtime>  
18\pgAdmin 4\runtime\psql.exe" "host=localhost po  
postgres sslmode=prefer connect_timeout=10" 2>>&1  
psql (18.4)  
WARNING: Console code page (437) differs from Win  
8-bit characters might not work correctl  
page "Notes for Windows users" for detai  
Type "help" for help.  
  
lab8shop=# SELECT id, name, price FROM products;  
 id | name | price  
-----+-----+-----  
  2 | Oppo find X5 (673380301-0 Sec 1) | 39900  
(1 row)  
  
lab8shop=#
```

ภาพที่ 7 ฐานข้อมูล PostgreSQL (pgAdmin 4) — ตาราง product_details

The screenshot shows the pgAdmin 4 interface. On the left, the 'product_details' table is selected, showing its columns and constraints. The columns are: id (bigint), description (character varying(255)), dimensions (character varying(255)), manufactured_country (character varying(255)), warranty (character varying(255)), and weight (double precision). The constraint is: product_details_pkey.

ภาพที่ 8 ฐานข้อมูล PostgreSQL (pgAdmin 4) — ตาราง reviews



▼	reviews
▼	Columns (6)
	id bigint
	comment character varying(255)
	rating integer
	review_date date
	reviewer character varying(255)
	product_id bigint
▼	Constraints (2)
	fkpl51cejpw4gy5swfar8br9ngi
	reviews_pkey